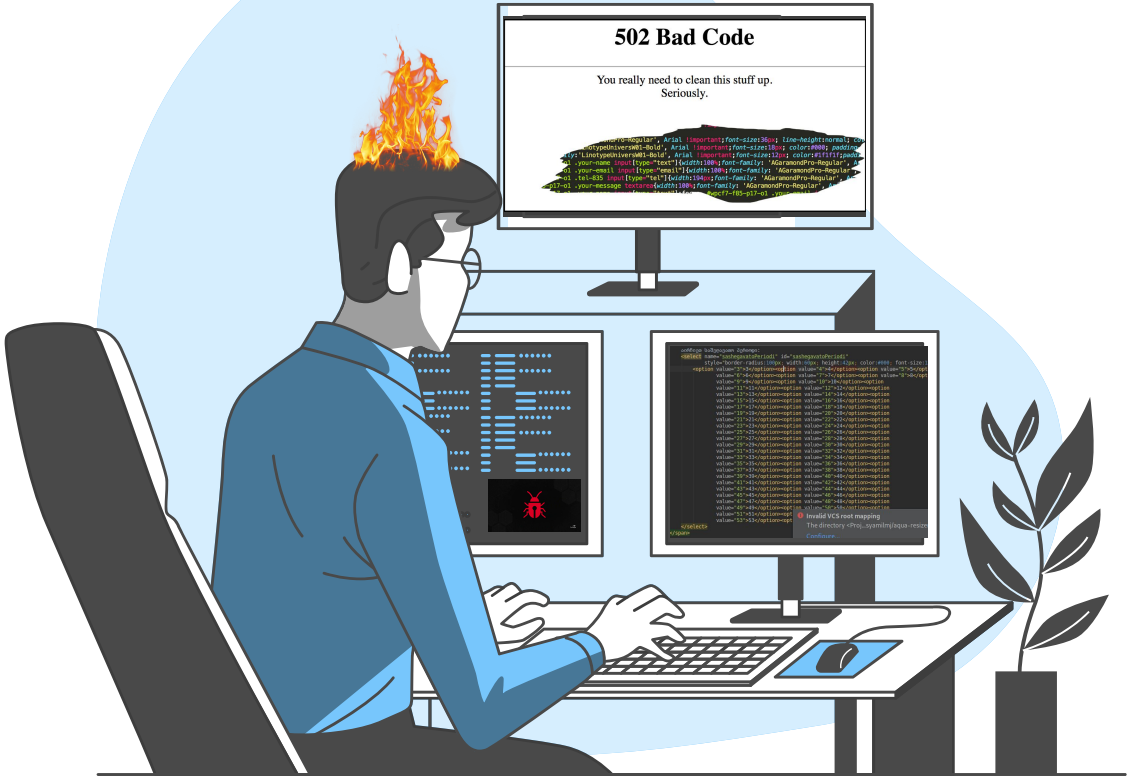


Anti Patterns

João Palma 14/04/23
POWrangers_81

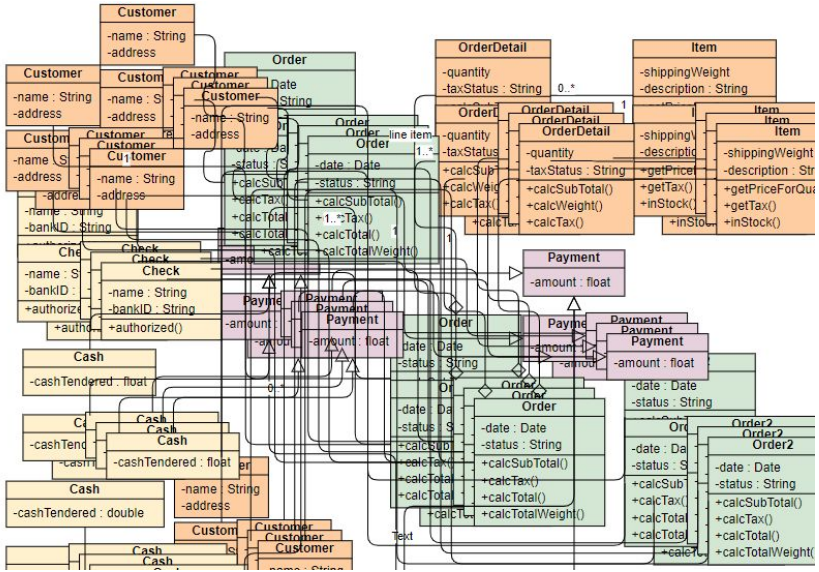
What is a Anti-Patern

- In software, anti-pattern is a term that describes how NOT to solve recurring problems in your code. Anti-patterns are considered bad software design and are usually ineffective or obscure fixes.

- They generally also add "technical debt" - which is code you have to come back and fix properly later.

1. Spaghetti Code

- Zero structure.
- Difficult to follow and tangled together (like spaghetti).
- Maintenance nightmare, near impossible to add new functionality.



Co-Worker: Let's create some well organized and structured code.

Me:

2. Golden Hammer / Leroy Merlin

- Doesn't quite fit what you need but gets the job done.
- Serious performance hit
- Takes twice as long to code up


```
public class GoldenHammer {

    public static void main(String[] args) {
        List<String> names = new ArrayList<>();
        names.add("John");
        names.add("Mary");
        names.add("Bob");
        names.add("Jane");
        names.add("Sara");

        for (String name : names) {
            if (name.equals("Bob")) {
                System.out.println("Hello, Bob!");
            } else if (name.equals("Mary")) {
                System.out.println("Hello, Mary!");
            } else {
                System.out.println("Hello, " + name + "!");
            }
        }
    }
}
  return go(f, seed, [])
}
```

3. Boat Anchor

- Is where programmers leave code in the codebase because they might need it later.
- Causes maintenance nightmares in the codebase that contains all that obsolete code
- Obsolete code makes your build time longer

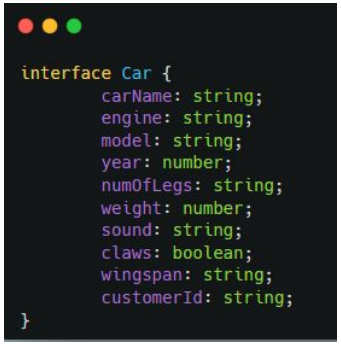
4. Dead code

- Dead code is similar to Boat Anchor
- Code written by someone who doesn't work at your company and there's a function that doesn't look like it is doing anything. But it is called from everywhere!
- You ask around and no-one else is quite sure what it's doing, but everyone's too worried to delete it.
- Monkey testing

(comment out to see what blew up)

5. God Object / TudoNaMain

- Everything in one object / class
- Has too much responsibility, functionality, and complexity
- It violates the Single Responsibility Principle (SRP) and can be difficult to maintain and test
- Programmers often compare this problem to asking for a banana, but you receive a gorilla holding a banana. You got what you asked for, but more than what you need.



```
interface Car {
        carName: string;
        engine: string;
        model: string;
        year: number;
        numOfLegs: string;
        weight: number;
        sound: string;
        claws: boolean;
        wingspan: string;
        customerId: string;
}
```

6. Magic Numbers

- Is a number in the code that seems arbitrary and has no context or meaning.
- One of the most important aspects of code quality is how it conveys intention.
- Magic numbers hide intention so they should be avoided.

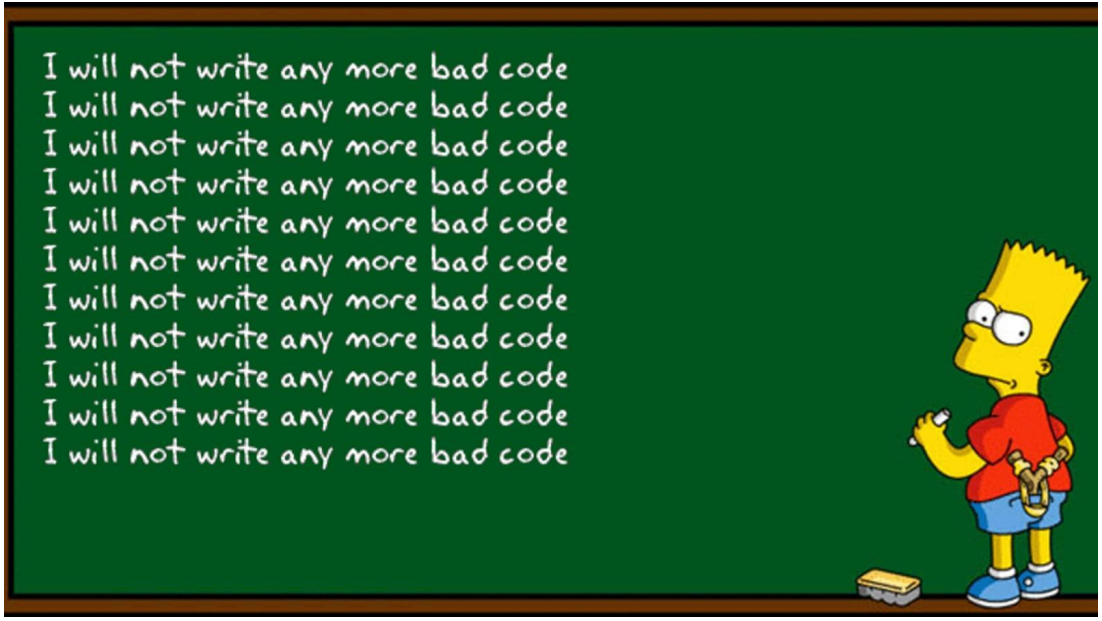
```
public class Circle {
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    public double calculateCircumference() {
        return 2 * 3.14 * radius; // Magic number: 3.14
    }
}
  return go(f, seed, [])
}
```

```
public class Circle {
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    public static final double PI = 3.14;

    public double calculateCircumference() {
        return 2 * PI * radius;
    }
}
```

Created a constant PI to represent the magic number 3.14.

This makes the code more readable and easier to maintain.

Thanks!


I will not write any more bad code
I will not write any more bad code
I will not write any more bad code
I will not write any more bad code
I will not write any more bad code
I will not write any more bad code
I will not write any more bad code
I will not write any more bad code
I will not write any more bad code
I will not write any more bad code